1. Mục tiêu

- Hiểu rõ vai trò và lợi ích của Docker Compose trong việc quản lý các ứng dụng đa container — mô hình gần với cách một hệ thống server thực tế được triển khai (backend + database + storage + reverse proxy...).
- Có khả năng viết file `docker-compose.yml` để định nghĩa và chạy toàn bộ hệ thống backend của nhóm (API, database, minio) chỉ bằng một lệnh, thay vì chạy `docker run` thủ công cho từng service như buổi trước.
- Biết cách lưu trữ dữ liệu bền vững (persistent) cho database bằng volume, và quản lý cấu hình/secret bằng file `.env`.
- Biết cách cấu hình để chạy nhiều instance của cùng một service (ví dụ: 2 backend từ 1 image) và hiểu đây là bước đầu của việc scale ứng dụng server theo chiều ngang (horizontal scaling).
- Nắm vững các lệnh Docker Compose cơ bản để vận hành, debug và dọn dẹp hệ thống nhiều container.
- Chuẩn bị tốt cho phần thuyết trình và demo trên hệ thống backend thực tế của nhóm.

2. Nội dung cần nghiên cứu và chuẩn bị

2.1. Giới thiệu Docker Compose

- **Docker Compose là gì?** Công cụ giúp định nghĩa và chạy các ứng dụng Docker đa container. Sử dụng file YAML để cấu hình các service, network, và volume.
- **Lợi ích:** Đơn giản hóa quá trình quản lý các ứng dụng phức tạp, dễ dàng tái tạo môi trường phát triển.

2.2. File `docker-compose.yml`

- **Cấu trúc cơ bản:**
  - `version`: Phiên bản của Docker Compose file format.
  - `services`: Định nghĩa các service (container) trong ứng dụng.
    - `build`: Đường dẫn đến Dockerfile hoặc context để build image.
    - `image`: Tên image để sử dụng (nếu không build).
    - `ports`: Ánh xạ cổng giữa host và container.
    - `environment` / `env_file`: Biến môi trường cho container, khai báo trực tiếp hoặc load từ file `.env`.
    - `volumes`: Ánh xạ volume để lưu trữ dữ liệu hoặc chia sẻ file.
    - `depends_on`: Định nghĩa sự phụ thuộc giữa các service (kèm điều kiện `condition: service_healthy` nếu dùng `healthcheck`).
    - `healthcheck`: Định nghĩa cách Docker kiểm tra service đã sẵn sàng (ví dụ: gọi thử một endpoint `/health` hoặc lệnh `pg_isready` cho database).
    - `restart`: Chính sách tự khởi động lại container khi bị crash (`no`, `always`, `on-failure`, `unless-stopped`).
    - `networks`: Định nghĩa network mà service tham gia.
  - `networks`: Định nghĩa các network tùy chỉnh.
  - `volumes`: Định nghĩa các volume để lưu trữ dữ liệu.
- **Ví dụ:** Viết `docker-compose.yml` để chạy một ứng dụng backend, một database (PostgreSQL/MySQL), và một MinIO server.

2.3. Quản lý dữ liệu và cấu hình

- **Volume (Named volume vs Bind mount):** Vì sao database chạy trong container cần volume để không mất dữ liệu khi container bị xóa/tạo lại. Named volume do Docker quản lý (phù hợp cho data của DB), bind mount ánh xạ trực tiếp tới thư mục trên host (phù hợp khi cần xem/chỉnh sửa file từ máy host).
- **File `.env` và biến môi trường:** Tách các giá trị cấu hình/nhạy cảm (mật khẩu database, secret key, connection string) ra khỏi `docker-compose.yml` bằng file `.env`, tránh commit thông tin nhạy cảm lên git (liên hệ `.gitignore`).
- **Service discovery giữa các container:** Trong cùng một network của Docker Compose, các service có thể gọi nhau bằng chính tên service (ví dụ backend kết nối database qua host `db` thay vì `localhost`), không cần biết IP nội bộ.

2.4. Multi-container và Multi-instance

- **Multi-container:** Chạy nhiều loại container khác nhau (backend, db, minio) cùng nhau trong một ứng dụng.
- **Multi-instance:** Chạy nhiều instance của cùng một service. Ví dụ: từ 1 image backend, build ra 2 container khác nhau, mỗi cái chạy ở một port riêng biệt.
  - Sử dụng `ports` khác nhau cho mỗi instance.
  - Sử dụng `scale` trong `docker compose up` để chạy nhiều instance.
  - **Vì sao cần multi-instance:** Đây là bước đầu của horizontal scaling — khi một ứng dụng server thật chịu nhiều tải, ta chạy nhiều bản sao (instance) của cùng một backend thay vì cố nâng cấp một server duy nhất (vertical scaling).
  - **Reverse proxy / load balancing (mở rộng):** Đặt một Nginx (hoặc service tương tự) phía trước các instance backend để phân phối request, giúp multi-instance thực sự có ý nghĩa với người dùng cuối thay vì chỉ chạy song song không ai gọi tới.
- **Phân biệt:**
  - **Image vs Container:** Image là bản thiết kế, container là thực thể chạy từ image.
  - **Build-time config vs Run-time config:** Build-time là các cấu hình trong Dockerfile (ví dụ: `ENV`, `ARG`), Run-time là các cấu hình khi chạy container (ví dụ: biến môi trường qua `-e` hoặc `environment` trong `docker-compose.yml`).

2.5. Các lệnh Docker Compose cơ bản

- **`docker compose up`:** Build (nếu cần) và khởi động tất cả các service được định nghĩa trong `docker-compose.yml`.
- **`docker compose up -d`:** Chạy các service ở chế độ detached (chạy ngầm).
- **`docker compose up -d --build`:** Build lại image trước khi khởi động (dùng khi có thay đổi code/Dockerfile).
- **`docker compose down`:** Dừng và xóa tất cả các container, network được tạo bởi `up`.
- **`docker compose down -v`:** Như trên, đồng thời xóa luôn các volume (xóa sạch dữ liệu database — cẩn thận khi dùng).
- **`docker compose ps`:** Liệt kê trạng thái của các service.
- **`docker compose logs [service_name]`:** Xem log của một hoặc tất cả các service.
- **`docker compose logs -f [service_name]`:** Theo dõi log real-time (follow).
- **`docker compose restart [service_name]`:** Khởi động lại một hoặc tất cả các service.
- **`docker compose build [service_name]`:** Build lại image cho một hoặc tất cả các service.
- **`docker compose exec [service_name] <command>`:** Thực thi lệnh bên trong một container của service.

- `docker compose config`**:** Kiểm tra và in ra cấu hình `docker-compose.yml` đã được resolve (hữu ích để debug lỗi cú pháp/biến môi trường).
- `docker compose scale <service_name>=<number>` (hoặc `up --scale <service_name>=<number>`): Chạy nhiều instance của một service.

2.6. Gợi ý tìm hiểu thêm (nâng cao): Giới hạn tài nguyên và giới hạn log cho từng service

Đây là hai vấn đề rất hay gặp khi vận hành container thật trong thời gian dài (không phải chỉ chạy demo vài phút), nên dù không bắt buộc đưa vào demo chính, sinh viên nên tìm hiểu để biết cách phòng tránh sự cố:

- **Giới hạn tài nguyên (CPU/Memory) cho từng service:** Mặc định một container trong `docker-compose.yml` không bị giới hạn CPU/RAM, có thể chiếm dụng gần như toàn bộ tài nguyên của máy host nếu có lỗi (memory leak, vòng lặp vô hạn...), ảnh hưởng đến các service khác đang chạy cùng máy. Tìm hiểu cách khai báo giới hạn ngay trong `docker-compose.yml`, ví dụ:

  ```
  services:
    backend:
      image: my-app
      deploy:
        resources:
          limits:
            cpus: "1.0"
            memory: 512M
          reservations:
            memory: 256M
  ```

  (Với Docker Compose chạy độc lập — không qua Swarm — có thể cần dùng thêm cú pháp tương đương `mem_limit`, `cpus` tùy phiên bản Compose; nên tự kiểm tra tài liệu chính thức để dùng đúng cú pháp với phiên bản đang cài.)
- **Giới hạn kích thước log của container:** Container chạy càng lâu, log (`stdout`/`stderr`) càng được Docker ghi lại càng nhiều, có thể khiến ổ đĩa của máy host bị đầy dần nếu không giới hạn (một lỗi phổ biến trong thực tế khi vận hành server dài ngày mà không để ý). Tìm hiểu cách cấu hình `logging driver` với giới hạn kích thước và số file log tối đa, ví dụ:

  ```
  services:
    backend:
      image: my-app
      logging:
        driver: "json-file"
        options:
          max-size: "10m"
          max-file: "3"
  ```

  Cấu hình trên giới hạn mỗi file log tối đa 10MB, giữ lại tối đa 3 file (log cũ hơn sẽ tự bị xoay vòng/xóa), tránh log phát triển không giới hạn theo thời gian.
- **Vì sao nên biết trước những cấu hình này:** Trong bài học chỉ demo trong thời gian ngắn nên khó thấy hậu quả, nhưng đây là hai nguyên nhân phổ biến gây sự cố (server bị đứng do hết RAM, hết dung

lượng ổ đĩa do log) khi vận hành hệ thống thật trong môi trường production — nên nắm được để áp dụng khi cần.

3. Chuẩn bị thuyết trình và demo

- **Slide:** Giới thiệu Docker Compose, cấu trúc file `docker-compose.yml`, quản lý dữ liệu/cấu hình, các khái niệm multi-container, multi-instance và các lệnh cơ bản.
- **Bắt buộc — gắn với hệ thống backend thực tế của nhóm:**
  - Viết file `docker-compose.yml` để chạy toàn bộ hệ thống của nhóm: backend (API), database (PostgreSQL/MySQL...), và MinIO (nếu ứng dụng có dùng object storage).
  - Dùng file `.env` để cấu hình các giá trị nhạy cảm (mật khẩu DB, secret...) thay vì hard-code trong `docker-compose.yml`.
  - Cấu hình `volumes` để dữ liệu database không bị mất khi container bị dừng/xóa (chứng minh bằng cách xóa container rồi tạo lại, dữ liệu vẫn còn).
  - Chạy hệ thống với `docker compose up -d` và kiểm tra các service hoạt động, gọi thử API thật kết nối đến database qua tên service (không phải `localhost`).
  - **Bắt buộc:** Từ 1 image backend, chạy ra 2 instance khác nhau, mỗi cái ở một port riêng biệt. Minh họa cách cấu hình trong `docker-compose.yml` và/hoặc chạy bằng `docker compose up --scale`.
  - **Khuyến khích:** Đặt một Nginx làm reverse proxy phía trước 2 instance backend để load balancing, chứng minh cả 2 instance đều nhận được request khi gọi lặp lại vào một địa chỉ duy nhất.
  - Sử dụng các lệnh `docker compose ps`, `logs -f`, `restart`, `down`.
  - Giải thích sự khác biệt giữa image và container, build-time và run-time config, và vì sao cần multi-instance trong quá trình demo.
- **Thời gian:** Phân bổ thời gian hợp lý cho lý thuyết và demo.

4. Phân công công việc và ước lượng thời gian (Gợi ý)

Task	Người thực hiện (Gợi ý)	Ước lượng (Point)
Nghiên cứu về Docker Compose và cấu trúc file	TV1	3
Nghiên cứu về volume, `.env`, healthcheck, restart policy	TV2	3
Nghiên cứu về Multi-container và Multi-instance	TV2	3
Chuẩn bị nội dung slide thuyết trình	Nhóm trưởng	2
Viết `docker-compose.yml` cho backend + db + minio, cấu hình volume và `.env`	TV3	5
Cấu hình và demo 2 instance backend (+ Nginx load balancing nếu làm)	TV1	5
Thực hành các lệnh Docker Compose và chuẩn bị demo	TV2, TV3	4
Tổng hợp, kiểm tra và chạy thử demo	Cả nhóm	2
Luyện tập thuyết trình và demo	Cả nhóm	1

Task	Người thực hiện (Gợi ý)	Ước lượng (Point)
Tổng cộng		**28**

5. Tài liệu tham khảo

- **Tài liệu chính thức:** Docker Compose Documentation (docs.docker.com/compose/), Docker Volumes, Docker Networking, Docker Compose `deploy.resources`, Docker Logging drivers (`json-file` options).
- **Bài viết:** Các bài hướng dẫn Docker Compose từ cơ bản đến nâng cao, đặc biệt về volume, healthcheck, load balancing với Nginx, giới hạn CPU/memory và giới hạn kích thước log cho container.
- **Video:** Các video hướng dẫn sử dụng Docker Compose để orchestrate multi-container applications, và cấu hình Nginx reverse proxy cơ bản.

6. Ghi chú quan trọng

- **Hiểu rõ luồng dữ liệu:** Khi làm việc với nhiều container, hãy hình dung rõ luồng dữ liệu giữa chúng (backend gọi database qua tên service, client gọi backend qua reverse proxy nếu có).
- **Sử dụng biến môi trường:** Tận dụng biến môi trường và file `.env` để cấu hình linh hoạt cho các service, không hard-code secret.
- **Không mất dữ liệu:** Luôn dùng volume cho database; cẩn thận khi chạy `docker compose down -v` vì lệnh này sẽ xóa luôn dữ liệu.
- **Kiểm tra log thường xuyên:** Log là công cụ quan trọng để debug khi có vấn đề.
- **Đừng để tài nguyên và log không giới hạn:** Container chạy càng lâu càng dễ gây sự cố nếu không giới hạn CPU/memory và kích thước log (mục 2.6) — đây là kiến thức nên biết trước khi đưa hệ thống ra vận hành thật, kể cả khi không bắt buộc demo trong buổi này.
- **Đây là bước chuẩn bị cho việc triển khai thật:** Một `docker-compose.yml` chạy đúng trên máy local chính là nền tảng để sau này deploy hệ thống backend của nhóm lên một server thật.

Chúc các bạn học tập hiệu quả và thành công!